Module-V
Lecture-I
Introduction and Basic Operations on Temporal Logic

1. **Introduction**

As discussed in the third module of the "Design" part of the course, any digital system can be represented by a collection of Boolean expressions. Each Boolean expression represents one output function/signal of the system. In the previous module (of the "Verification" part of the course) we have discussed one data structure by which we can represent the Boolean function in an efficient way. It is also mentioned that ROBDD (Reduced Order Binary Decision Diagram) provides the canonical representation of a Boolean function. Due to this, the checking for validity, satisfiability of Boolean expressions has become easy. During the design process, we move through different levels of abstractions, starting from very high level to detailed level. While moving from one level to other level of abstraction, sometime it is needed to check for their equivalence. BDD representation provides an efficient method for equivalence checking mainly at the Boolean level.

During the design phase, it is always not required to go for Boolean expression for each function. We may go for design at the functional level and represent the whole system with some abstracted model. But while going through the design phases, it is better to check for correctness of design in every stage. While conceiving the idea of a digital system, we know the system should meet some specifications or properties. While designing the system, we must always ensure that the design will always meet those specification or requirements. One way of checking for the correctness of specification is logical reasoning. We may use some logical formalism to represent the specification and use the underlying theory of that logical framework to reason about it. In this lecture we will look for some logical framework by which we can formally represent the specification. We will start with propositional logic and predicate logic. Then we will see why we need some other logic to capture the specifications.

The aim of logic in computer science is to develop languages to model the situations we encounter as computer science professionals, in such a way that we can reason about

them formally. Reasoning about situations means constructing arguments about them; we want to do this formally, so that the arguments are valid and can be defended rigorously, or executed on a machine.

2. **Propositional Logic**

In order to make arguments rigorous, we need to develop a language in which we can express sentences in such a way that bring out their logical structure. The language we begin with is the language of propositional logic. It is based on propositions, or declarative sentences which one can, in principle, argue as being true or false. Examples of declarative sentences are:

(1) The sum of the numbers 5 and 10 equals 15.

(2) Bhim reacted violently to Duryodhan's accusations.

(3) Every even natural number greater than 2 is the sum of two prime numbers.

(4) All students like sauce on their bread.

We assign certain distinct symbols *p, q, r*. . . or sometimes *p1, p2, p3,* . . . to each of these atomic sentences and we can then code up more complex sentences in a compositional way. For example, given the atomic sentences

p: 'I qualified GATE exam in 2010'

q: 'I prepared well'

r: 'I got a non-scholarship seat in IIT Guwahati '

we can form more complex sentences according to the rules below:

$\neg$**:** The negation of *p* is denoted by $\neg p$ and expresses 'I did not qualified GATE exam in 2010,' or equivalently 'It is not true that I qualified GATE exam in 2010.'

$\vee$**:** Given *p* and *r* we may wish to state that at least one of them is true: 'I qualified GATE exam in 2010, or I got a non-scholarship seat in IIT Guwahati'; we denote this declarative sentence by $p \vee r$ and call it the disjunction of *p* and *r*.

$\wedge$**:** Dually, the formula $p \wedge r$ denotes conjunction of *p* and *r*: 'I qualified GATE exam in 2010 and I got a non-scholarship seat in IIT Guwahati.' (may be his score was not high!!!)

$\rightarrow$**:** 'If I qualified GATE exam in 2010', then I prepared well.' expresses an implication between *p* and *q*, suggesting that *q* is a logical consequence of *p*. We write $p \rightarrow q$ for that. We call *p* the assumption of $p \rightarrow q$ and *q* its conclusion.

2.1 The Need for a Richer Language:

The propositional logic is not powerful enough to represent all types of assertions that are used in computer science and mathematics, or to express certain types of relationship between the propositions such as equivalence.

For example, the assertion '*x* is greater than 1', where *x* is a variable, is not a proposition because you cannot tell whether it is true or false unless you know the value of *x*. Thus, propositional logic cannot deal with such sentences. However, such assertions appear quite often in mathematics and we want to do inferencing on those assertions.

Also the pattern involved in the following logical equivalences cannot be captured by the propositional logic:

'Not all objects that glitter is gold' is equivalent to 'Some objects that glitter are not gold'.

'Not all integers are prime' is equivalent to 'Some integers are not prime'

'Not all cars are driven by petrol' is equivalent to 'Some cars are not driven by petrol'

Each of the above propositions are treated independently of the others in propositional logic. For example, if *P* represents 'Not all objects that glitter is gold' and Q represents 'Some integers are not prime', then there is no mechanism in propositional logic to find out whether or not *P* is equivalent to *Q*. Hence, to be used in inferencing, each of these equivalences must be listed individually rather than dealing with a general formula that covers all these equivalences collectively and instantiating these become necessary, if only propositional logic is used.

Thus we need more powerful logic to deal with these and other problems. Predicate logic is one type of such logic family.

3. **Predicate Logic**

A predicate is a generalization of a propositional variable. Suppose we have three propositions

R: 'He is working hard',

U: 'He gets good marks', and

W: 'He gets fail grade'.

Suppose, further we have three hypotheses or expressions that we assume are true:

$R \rightarrow U$ 'If he is working hard, then he gets good marks',

$U \rightarrow \neg W$ 'If he gets good marks, then he doesn't get fail grade', and

$\neg R \rightarrow \neg U$ 'If he is not working hard, then he doesn't get good marks'

What is true for "He" is also true for Ram, and Shyam, and Madhu, and so on. We can define symbol *U* to be a predicate that takes an argument *X*.

The expression:

U(X): 'X gets good marks'

Possibly, for some values of *X, U(X)* is true, and for other values of *X, U(X)* is false. Similarly,

W(X): 'X gets fail grade."

R(X): "*X* is working hard",

In predicate logic, we also use two quantifiers, one is universal quantifier, for all ($\forall$), and the other is existential quantifier, there exists ($\exists$).

We can write: $\forall x\ [R(x) \rightarrow U(x)]$ for all those who work hard will get good marks.

Also, $\exists x\ [U(x)]$ There exists someone who gets good marks.

Propositional/predicate logic, can represent statements whose truth value is constant in time. However, there are statements and we need to reason about them whose truth values change over time, e.g., there is peace in the country. This truth of the statement may vary with time. To represent such statements we need a more powerful logic namely, temporal logic.

4. **Temporal Logic**

The idea of temporal logic is that a formula is not *statically* true or false in a model, as it is in propositional and predicate logic. Instead, the models of temporal logic contain several states and a formula can be true in some states and false in others.

Thus, the static notion of truth is replaced by a *dynamic* one, in which the formulas may change their truth values as the system evolves from state to state.

In a temporal logic we can express statements like:

"I am *always* happy",

"I will *eventually* be happy", or

"I will be happy *until* I do something wrong"

"I am happy."

Consider the statement: "I am happy." Though its meaning is constant in time, the truth value of the statement can vary in time. Sometimes the statement is true, and sometimes the statement is false, but the statement is never true and false simultaneously. In temporal logic, statements can have a truth value which can vary in time. Contrast this with a propositional/predicate logic, which can only discuss statements whose truth value is constant in time.

Temporal logic always has the ability to reason about a time line. Linear temporal logic is restricted to reasoning about a time line. Branching logics, however, can reason about multiple time lines. To continue with the example, in a branching logic we may state that "there is a possibility that I will stay happy forever." We may also state that "there is a possibility that eventually I am no longer happy." If we do not know whether or not I will ever do something wrong, these statements are both true.

Temporal logic can be classified according to their particular perception of “time”.

- *Linear time* logic thinks of time as a set of paths, where a path is a sequence of time instances.
- *Branching-time* logic represents time as a tree, rooted at the present instance and branching out into the future.

4.1 **Temporal Operators:**

Temporal logic has two kinds of operators namely, (i) logical and modal and (ii) temporal. Logical operators are usual truth-functional operators ($\neg, \vee, \wedge, \rightarrow$) (used for propositional and predicate logic). The basic temporal operators are of two types namely (i) future and (ii) past; the details are as follows.

Operator	Textual	Explanation
Future Operators		
○	**X** φ	**neXt:** φ holds at the next state.
◊	**F** φ	**Future**: φ eventually holds (somewhere on the subsequent path).
□	**G** φ	**Globally:** φ holds on the entire subsequent path.
U	φ **U** ψ	**Until:** ψ holds at the current or a future position, and φ has to hold until that position. At that position φ does not have to hold any more.
R	φ **R** ψ	**Release:** φ releases ψ if ψ is true until the first position in which φ is true (or forever if such a position does not exist).
Past Operators		
●	●φ	**Previous:** φ has to hold in the previous state.
	φ	**Eventually in past:** φ eventually has to hold in the past.
■	■φ	**Globally in past:** φ has to hold on the entire previous path.
β	φ β ψ	**Back to:** φ holds in all previous positions (including the present) starting at the last position ψ held.

Temporal formulas are interpreted over a model, which is an infinite sequence of states. Given a model M and a temporal formula φ, we define an inductive definition for the notion of φ holding at a position S_j for $j \geq 0$ in M and denoted by $(M, S_j) \models \varphi$. The inductive definitions are discussed as follows.

A. Next: φ has to hold at the next state.

Symbol:

$X\varphi$

Definition:

$$(M, S_j) \models X\varphi \Leftrightarrow (M, S_{j+1}) \models \varphi$$

Example:

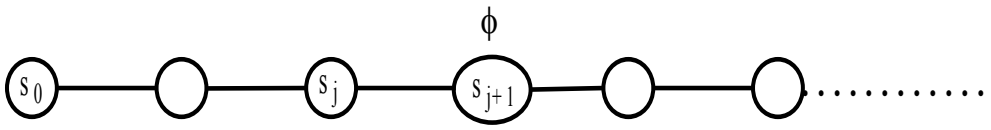


Figure 1. Example of $X\varphi$

In Figure 1, state S_j satisfies $X\varphi$ as its next state S_{j+1} satisfies φ .

B. Future: φ eventually has to hold (somewhere on the subsequent path).

Symbol:

$\boldsymbol{F}$ φ

Definition:

$$(M, S_j) \models F\varphi \Leftrightarrow \exists k, k \geq j, (M, S_k) \models \varphi$$

Example:

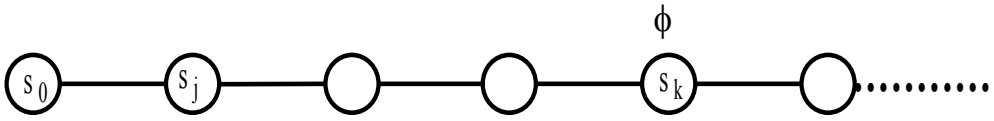


Figure 2. Example of $F\varphi$

In Figure 2, state S_j satisfies $F\varphi$ as future state S_k satisfies φ .

C. Globally: φ has to hold on the entire subsequent path.

Symbol:

G φ

Definition:

$$(M, S_j) \models G\varphi \Leftrightarrow \forall k, k \geq j, (M, S_k) \models \varphi$$

Example:

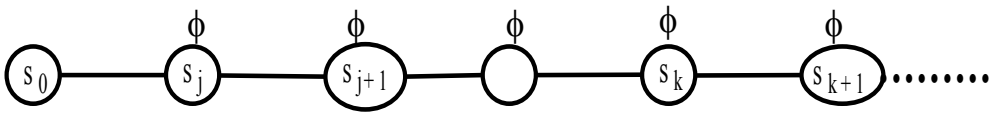


Figure 3. Example of **G** φ

In Figure 3 S_j satisfies **G** φ as all future states satisfies φ.

D. Until : ψ holds at the current or a future position, and φ has to hold until that position. At that position φ does not have to hold any more.

Symbol:

(φ U ψ)

Definition:

$$(M, S_j) \models \Psi U \varphi \Leftrightarrow \exists k, (M, S_k) \models \Psi \text{ and } \forall k, j < k (M, S_j) \models \varphi$$

Example:

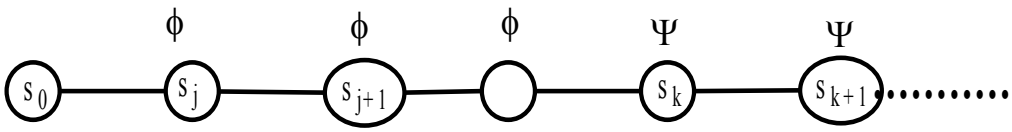


Figure 4. Example of **(φ U ψ)**

In Figure 4, S_j satisfies **(**φ **U** ψ) because φ is true for all states S_j $j<k$ and then ψ is true for state S_k .

E. Release: φ releases ψ if ψ is true until the first position in which φ is true (or forever if such a position does not exist).

Symbol:

(φ R ψ)

Definition:

$$(M, S_j) \models \varphi R \Psi \Leftrightarrow \forall j, j \le k (M, S_j) \models \Psi \text{ and } (M, S_k) \models \varphi$$
$$\text{OR } \forall j (M, S_j) \models \Psi$$

Example

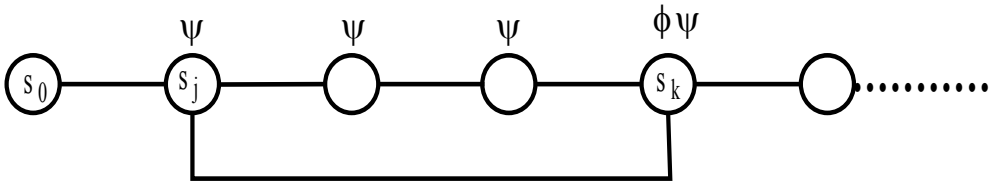


Figure 5. Example of (φ R ψ)

In Figure 5, S_j satisfies **(**φ **R** ψ) because ψ is true for all states S_j *through* S_k and then ψ is true for states starting from S_k .

F. Previous: φ has to hold at the previous state.

Symbol:

●φ

Definition:

$$(M, S_j) \models \bullet\varphi \Leftrightarrow \exists (M, S_{j-1}) \models \varphi$$

Example:

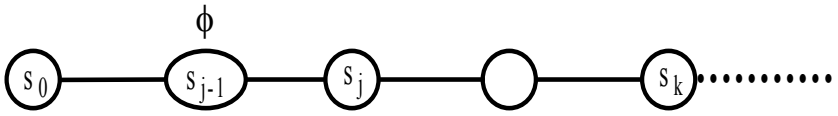


Figure 6. Example of ●φ

In Figure 6, state S_j satisfies •φ as its previous state S_{j-1} satisfies φ .

G. Eventually in past: φ eventually has to hold in the past.

Symbol:

φ

Definition:

$$(M, S_j) \models \quad \varphi \Leftrightarrow \exists k, k \le j (M, S_k) \models \varphi$$

Example:

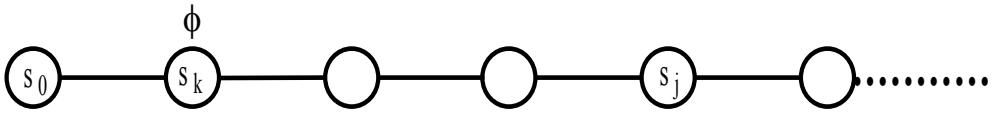


Figure 7. Example of φ

In Figure 7, state S_j satisfies φ as eventually a past state S_k satisfies φ.

H. Globally in past: φ has to hold on the entire previous path.

Symbol:

■φ

Definition:

$$(M, S_j) \models \blacksquare \varphi \Leftrightarrow \forall k, k \le j (M, S_k) \models \varphi$$

Example:

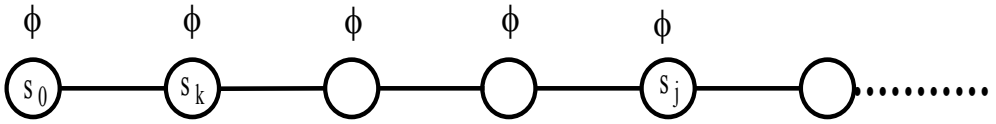


Figure 8. Example of ■φ

In Figure 8, state S_j satisfies ■φ, as globally in all past states staring backward from S_j, satisfies φ.

I. Back to: φ holds in all previous states (including the present) starting at the last position ψ held.

Symbol:

(φ *β* ψ)

Definition:

$$(M, S_j) \models \varphi \beta \Psi \Leftrightarrow \exists k (M, S_k) \models \Psi \text{ and } \forall j \ge k (M, S_j) \models \varphi \text{ until present state}$$

$$\text{OR } (M, S_j) \models \varphi \text{ for } j = 0 \text{ to present state}$$

Example

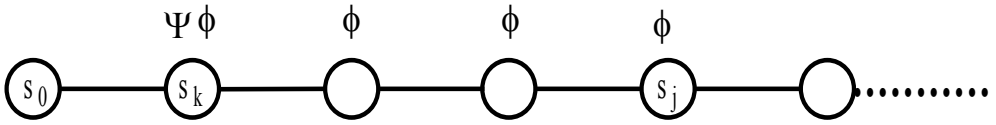


Figure 9. Example of **(φ *β* ψ)**

In Figure 9, in state S_k ψ is true and for all $j \geq k$ the states satisfy φ until present state S_j.

Now, we give some more examples of the temporal formulas.

4.2 Examples temporal formulas

A. (*P*→ *FQ*): If *P* is true in a state then in a future state *Q* is true.

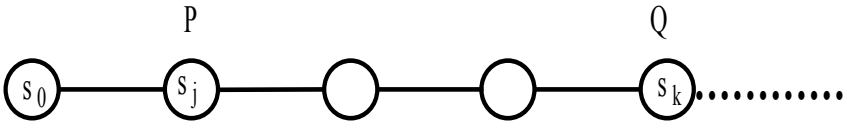


Figure 9. $P \rightarrow FQ$ holds in state S_j

For state S_j, $P \rightarrow FQ$ is true because *P* is true at S_j and for future state S_k, *Q* is true.

In state S_0, $P \rightarrow FQ$ is also vacuously true --If P is not true at S_0 then there is no need for any constraints.

B. ***(P $\vee$ XQ):*** Either P holds in a state or in next state Q holds

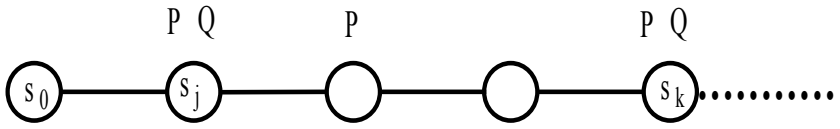


Figure 10. (P $\vee$ XQ) holds in all sates

As shown in Figure 10, **(P $\vee$ XQ)** is true for all states because either P is true or XQ (i.e., Q holds in next state) is true for all states.

C. ***(P $\vee$ (Q U R):*** Either P holds in a state or Q U R (Q until R) holds

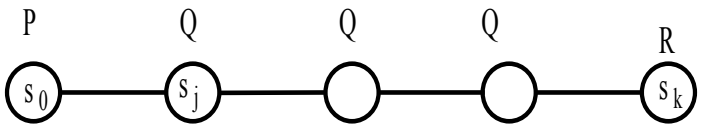


Figure 11. (P $\vee$ (Q U R) holds in all states

Here $P \vee (Q \cup R)$ is true for all states because P is true for S_0 and for others $(Q \cup R)$ is true.

D. ***(P $\wedge$ (Q U R)*:** P holds in a state and also Q U R (Q until R) holds in the state

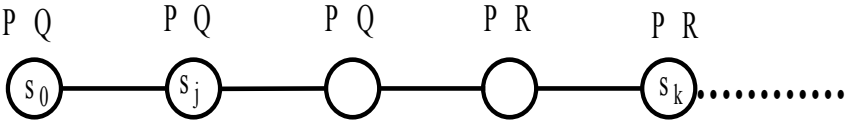


Figure 12. $(P \wedge (Q \cup R)$ holds in all sates

Here $P \wedge (Q \cup R)$ is true for all states because P is true for all state and (Q U R) is also true for all the states.

E. ***(P ^ •Q):*** *P* holds in a state and in the previous state *Q* holds

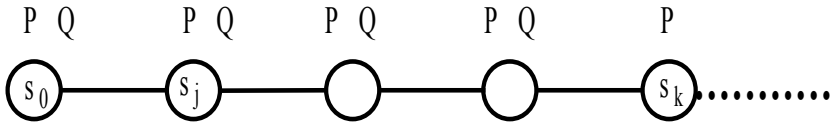


Figure 13. *(P ^ •Q)* holds in all sates

Here every state satisfy *P* and •*Q* (i.e., Q holds in the previous state).

5. **Conclusions**

In this lecture we have introduced that verification of high level system descriptions require logical reasoning about systems. For such reasoning we need to represent specifications using some logical paradigms. We have seen that depending upon requirements, different logic paradigms are required namely, propositional logic, predicate logic, temporal logic etc. Temporal logic paradigm, which is used to capture the timing modalities like next, future, part, etc. was discussed in detail. Temporal logic framework can be divided into different classes based on their expressive power namely, Linear-time Temporal Logic (LTL), Computational Tree Logic (CTL), CTL* etc. [1]. In the next lecture we will discuss in detail about CTL framework.

References

[1]. Michael Huth and Mark Ryan, “Logic in Computer Science: Modelling and Reasoning about Systems”, 2nd edition, Cambridge University Press, New York, NY, USA.

Question and Answers

Question: What does the temporal formula $(P \rightarrow Q)$ mean? Give an example where this formula is valid in all the states.

Answer: $(P \rightarrow Q)$ means that "If P holds in a state then eventually in past Q holds". In the example given below, $(P \rightarrow Q)$ holds in all states because (i) in all states except S_k, $(P \rightarrow Q)$ is vacuously true because P does not hold, (ii)At state S_k, P is true and eventually in past Q holds at S_j.

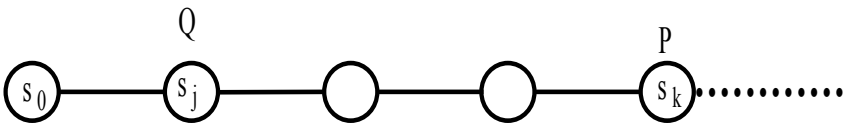


Figure 14. *(P→ Q)* holds in all sates